

# Prediction-Based Compression Using Large Language Models

Jean-Francois Chamberland\*, Krishna Narayanan\*, Khac-Hoang Ngo<sup>†</sup>, Giuseppe Durisi<sup>‡</sup>, Vishnu Teja Kunde\*,  
Chandra Shekhara Kaushik Valmeekam\*

\*Department of Electrical and Computer Engineering, Texas A&M University, College Station, TX, USA  
Emails: {krn, chmbrlnd, vishnukunde, vcskaushik9}@tamu.edu

<sup>†</sup>Department of Electrical Engineering, Linköping University, Linköping, Sweden  
Email: khac-hoang.ngo@liu.se

<sup>‡</sup>Department of Electrical Engineering, Chalmers University of Technology, Gothenburg, Sweden  
Email: durisi@chalmers.se

**Abstract**—Modern networked systems carry increasingly heterogeneous traffic, creating a demand for new designs that seamlessly integrate computing, inference, and timely information delivery. The fundamental compression limits for certain sources are well understood, and algorithms capable of asymptotically approaching these limits can be found in the literature. Nevertheless, recent advances in artificial intelligence and the widespread availability of powerful machine learning models make this an opportune moment to revisit the foundations of compression and reexamine key paradigms such as predict-then-code as building blocks for modern communication systems. In this context, this tutorial bridges established concepts in information theory with recent developments in machine learning to develop compression schemes that not only perform well but also exploit the modalities readily available on contemporary devices. The article begins with a brief overview of classical compression schemes such as Huffman coding and the Lempel–Ziv algorithms. It then quickly pivots to the predictive coding framework, in which a large language model (LLM) or a generative pre-trained transformer (GPT) serves as a next-token predictor. This predictive capability can be turned into a pragmatic compression scheme by coupling the output distributions with arithmetic coding. When both the source and destination have access to the same machine learning model, the original message can be recovered reliably at the destination. In certain practical scenarios, the proposed scheme outperforms existing approaches and provides new empirical estimates of the entropy rate of the English language. Interestingly, the predict-then-code paradigm can also provide advantages in systems where timely information delivery is a critical design objective. Furthermore, prompting or tuning the underlying machine learning model enables personalized compression, enhancing performance in transient or short-lived connections. Taken together, these perspectives suggest that learning-based compression may become a cornerstone of future networked systems, one that unifies inference, communication, and adaptation under a common predictive framework.

## I. INTRODUCTION

There are close connections between learning, prediction, and compression. Learning can be viewed as the process of discovering statistical structure in data, prediction as exploiting that structure to anticipate future outcomes, and compression as representing that same structure efficiently with minimal redundancy. An algorithm that learns a good model of a source can predict its future symbols with low uncertainty and, by

extension, encode them using fewer bits. This perspective provides both a theoretical foundation and a practical bridge between modern machine learning methods and classical information theory.

In his seminal 1951 work, Claude Shannon used human subjects to play a kind of *guessing game* to estimate the entropy of the English language [1]. At each step, a participant attempted to predict the next character given the preceding context, and Shannon translated the quality of these predictions into an empirical estimate of compressibility. Today, large language models (LLMs) act as our digital subjects, playing the same game at a scale and precision that was previously unimaginable. By viewing these models not just as text generators, but as probability estimators for sequential data, we can revisit classical questions about compression limits through a modern lens.

The connection between prediction and compression is fundamental: a predictor that assigns accurate conditional probabilities to the next symbol can be turned into a compressor with small redundancy, because likely outcomes can be described with fewer bits. Conversely, any lossless compressor implicitly defines a predictive distribution on messages. This prediction–compression duality has been exploited across numerous fields, including information theory, statistical inference, universal prediction, and online learning. Within the context of lossless text compression, the combination of a language model with arithmetic coding emerged early as a powerful realization of this connection [2]. The performance of such a scheme depends critically on the quality of the predictor, and every advance in predictive modeling invites a renewed examination of its impact on compression. For example, Goyal, Tatwawadi, Chandak, and Ochoa employed recurrent neural networks (RNNs) as predictors and demonstrated improved compression performance for certain sources [3].

Modern LLMs such as ChatGPT, Gemini, and LLaMA make this connection tangible. At their core, these models estimate high-dimensional conditional distributions for the next token given a context window, capturing dependencies that extend beyond classical  $n$ -gram or fixed-order Markov assumptions [4], [5]. From an information-theoretic standpoint, this

shift reflects a move from hand-crafted statistical models to learned representations that absorb broad regularities from data and exploit them for prediction. When a predictor becomes strong enough, its outputs can be interpreted as a practical proxy for the source statistics that govern compressibility.

A key theme of this article is that the *predict-then-code* architecture offers a clean separation of concerns. The predictor (classical, universal, or learned) supplies sequential probability assignments, while arithmetic coding provides a near-optimal mapping from those probabilities to a compressed bitstream. This modularity makes the connection between learning and coding especially transparent: improvements in predictive log-loss translate directly into reductions in achieved compression rate, often reported in bits per character (bpc). In this sense, compression becomes a rigorous and model-agnostic way to quantify the quality of a predictor.

This tutorial builds on these observations and explores how modern predictive models, particularly LLMs, can be integrated into compression frameworks. We begin by revisiting the fundamental principles that connect entropy rate and compression, and we review canonical schemes, including Huffman coding and Lempel–Ziv methods, that exploit known or learned structure in sequential sources. We then study arithmetic coding as a unifying tool for implementing *predict-then-code* schemes and for translating conditional probabilities into compressed representations. With this foundation in place, we examine how LLMs can be repurposed as powerful general-purpose predictors inside compression pipelines, and how this perspective yields new empirical estimates of the entropy rate of English. Our goal is to provide both conceptual clarity and practical directions to the reader interested in leveraging learned predictors as compressors in contemporary machine learning and information-theoretic settings.

## II. PREDICTION, COMPRESSION, AND THE HIDDEN STRUCTURE OF DATA

This section develops a foundational viewpoint from information theory: *structure*, *prediction*, and *compression* are not merely related concepts but different facets of the same underlying phenomenon. Data is compressible only to the extent that it possesses exploitable regularities, and the act of reducing description length is, at its core, an act of prediction. When a source exhibits statistical structure, past observations constrain future outcomes, enabling a predictor to assign a biased conditional distribution to the next symbol. A coding scheme can then translate these probability assignments into shorter representations. By contrast, when a source is effectively memoryless and close to uniform (so that the next symbol remains unpredictable even given the past), no lossless method can significantly outperform a symbol-by-symbol description. This perspective positions compression not merely as a file-handling utility, but as a rigorous, quantitative lens through which we can evaluate how well a model has captured a source’s statistical structure.

A central quantity that formalizes these ideas is the *entropy rate*  $H(\mathbf{S})$ , which represents the source’s irreducible uncertainty and thus the ultimate limit of lossless compression.

While the entropy of an isolated symbol measures marginal randomness, the entropy rate quantifies the residual uncertainty that remains after *all* statistical dependencies, both local and long-range, have been exploited. For structured sources such as natural language, the entropy rate can be substantially lower than the entropy of the marginal distribution precisely because context heavily constrains future symbols.

In the compression paradigm, the entropy rate functions as a non-constructive fundamental limit: it defines the intrinsic information density of the source, independent of encoding mechanism, dictionary size, or modeling choice. Practical algorithms such as Huffman or Lempel–Ziv provide achievable rates and therefore empirical upper bounds on this limit, but the limit itself is a property of the source alone. Consequently, shrinking the gap between an achieved rate and the entropy rate is a direct measure of how well a predictive model has learned the source. The remainder of this section makes these connections precise and sets the stage for later sections, where learned predictors, including LLMs, serve as probability engines inside modern compression pipelines.

### A. Entropy as Irreducible Uncertainty

Consider a stochastic process  $\mathbf{S} = \{S_1, S_2, \dots\}$  taking values in a finite alphabet  $\mathcal{S}$ , representing a source that generates symbols sequentially. The entropy of a single random variable  $S$  with probability mass function (PMF)  $p_S(s)$  is defined as

$$H(S) = - \sum_{s \in \mathcal{S}} p_S(s) \log_2 p_S(s) \quad (1)$$

and it measures the intrinsic uncertainty associated with a single draw from the source.

When the source produces symbols in an independently and identically distributed (iid) manner, the entropy  $H(S)$  characterizes the average number of bits required to describe each symbol in an optimal lossless representation. In this special case, entropy has a sharp operational meaning: no lossless compression scheme can, on average, represent the source using fewer than  $H(S)$  bits per symbol, while known schemes can approach this limit arbitrarily closely.

More generally, real-world sources such as natural language exhibit strong statistical dependencies across time. In such settings, the uncertainty of future symbols depends on the past, and single-letter entropy no longer captures the full structure of the source. This motivates the notion of entropy rate. For a general stochastic process  $\mathbf{S}$ , the entropy rate is defined as

$$H(\mathbf{S}) = \lim_{n \rightarrow \infty} \frac{1}{n} H(S_1, S_2, \dots, S_n), \quad (2)$$

provided the limit exists. The entropy rate quantifies the average uncertainty per symbol when the entire past of the sequence is taken into account. Intuitively,  $H(\mathbf{S})$  measures how predictable the source is in the long run. A highly structured source, in which the past strongly constrains the future, has a low entropy rate. Conversely, if even long histories leave substantial uncertainty about the next symbol, then the entropy rate is high. In this sense, entropy rate provides a measure of structure: lower entropy rate means less conditional uncertainty, and therefore greater predictability.

A foundational result that formalizes this interpretation is the Shannon–McMillan–Breiman theorem [6]. Under mild conditions such as stationarity and ergodicity, the theorem characterizes (almost surely) convergence to the entropy rate:

$$-\frac{1}{n} \log_2 p_{\mathbf{S}_{1:n}}(S_1, \dots, S_n) \rightarrow H(\mathbf{S}) \quad \text{as } n \rightarrow \infty, \quad (3)$$

where  $p_{\mathbf{S}_{1:n}}(S_1, \dots, S_n)$  denotes the probability assigned by the true source distribution to the observed length- $n$  sequence. In words, the normalized negative log-likelihood of a typical sequence converges to the entropy rate of the source. This result provides a bridge between probability, prediction, and compression: typical sequences concentrate on a set whose effective size is governed by  $2^{nH(\mathbf{S})}$ , and any compressor that assigns too few bits to represent such sequences will fail.

We emphasize that entropy and entropy rate are intrinsic properties of the source, not of any particular compression algorithm. They describe how much uncertainty remains after all statistical regularities have been taken into account, independent of how symbols are ultimately represented as bits. In this sense, entropy characterizes the structure of the data itself, rather than the mechanics of any specific encoding procedure. Compression algorithms, ranging from classical schemes such as Huffman coding to more sophisticated modern methods, can be viewed as mechanisms for approaching these fundamental limits. Their performance reflects how effectively they exploit structure in the source, but the limits themselves are dictated by the source statistics.

This distinction becomes especially important in modern settings where learning-based predictors are embedded within compression systems. Improvements in compression performance should not be interpreted merely as better encoders, but rather as evidence that the underlying predictive model has captured additional structure in the data. Seen this way, compression provides a principled and quantitative means of evaluating predictive quality.

### B. Entropy, Prediction, and Estimation

In practice, modern compressors rarely operate directly on individual characters. Instead, text is typically parsed into variable-length tokens, such as words, subwords, or byte-pair encodings [7]. While this tokenization step changes the representation of the source, it does not alter its underlying information content. Each token may correspond to a different number of characters, yet when averaged over long sequences, the number of bits required to describe the text must still reflect the same intrinsic uncertainty of the original source. As a result, compression rates expressed in bits per token can be translated into bits per character, enabling meaningful comparisons across models and representations.

A central insight from information theory is that entropy rate can be estimated empirically through prediction. When a model assigns probabilities to each next symbol (or token) in a sequence, the average negative log-probability assigned to the observed data is the *coding rate achieved under that model*. If the model is mismatched to the true source, this achieved rate upper bounds the true entropy rate. Intuitively, a model that consistently assigns high probability to the correct

continuation has captured much of the source’s structure and therefore yields a low entropy estimate. Conversely, inaccurate predictions translate directly into higher estimated entropy. In this sense, entropy provides a formal answer to a foundational question: *how much structure is available for exploitation?* The following subsections show how this question has been approached historically, from Shannon’s early experiments to modern probabilistic predictors, and why contemporary LLMs offer a new empirical handle on this elusive rate.

To make this connection precise, it is useful to distinguish three closely related quantities: entropy, cross-entropy, and relative entropy. Consider a stationary and ergodic source, let  $P$  denote the true (unknown) distribution of the next symbol given its past, and let  $Q$  denote the distribution produced by a predictor. If the source is actually distributed according to  $P$  but we encode symbols using probabilities  $Q$ , then the expected number of bits per symbol becomes the *cross-entropy*

$$H(P, Q) = \mathbb{E}_{S \sim P} [-\log_2 Q(S)]. \quad (4)$$

This is exactly the quantity measured by average log-loss in machine learning (and, after exponentiation, by perplexity [8], [9]). It captures the operational cost of using a mismatched model for compression. Cross-entropy admits the following decomposition:

$$H(P, Q) = H(P) + D(P \| Q), \quad (5)$$

where  $H(P) = \mathbb{E}_{S \sim P} [-\log_2 P(S)]$  is the entropy and

$$D(P \| Q) = \mathbb{E}_{S \sim P} \left[ \log_2 \frac{P(S)}{Q(S)} \right] \geq 0 \quad (6)$$

is the Kullback–Leibler (KL) divergence. In words,  $D(P \| Q)$  is the *price of mismatch*; it is the extra number of bits incurred, on average, when compressing data generated by  $P$  using the model  $Q$ . This perspective is central to machine learning, because modern language models are trained by minimizing cross-entropy, and compression performance improves precisely as their predictions reduce this mismatch penalty.

This connection between prediction and entropy estimation has a long history. As mentioned in Section I, early experiments by Shannon relied on human predictions to estimate the entropy of English text, and subsequent work formalized these ideas using probabilistic models and asymptotic arguments. Classical results show that, under mild assumptions, the average coding cost produced by any probabilistic predictor converges almost surely to an upper bound on the entropy rate of the source. Crucially, this bound holds regardless of the specific form of the predictor, provided it assigns valid probability distributions. When paired with an appropriate coding scheme, such predictive models can be turned directly into compressors whose performance approaches these entropy-based limits.

Modern language models make this connection concrete. They are trained by minimizing average negative log-likelihood (equivalently, cross-entropy) on token sequences. From a coding perspective, this objective has a direct operational meaning: if a model supplies conditional probabilities

$q_i(\cdot)$  and the realized sequence is encoded with an entropy coder, then the resulting description length satisfies

$$L(\mathbf{x}_{1:n}) \approx - \sum_{i=1}^n \log_2 q_i(x_i), \quad (7)$$

up to lower-order effects due to finite precision and termination conventions. In this sense, training loss is not merely correlated with compression performance; it is an explicit estimate of the number of bits required to describe the data under the model. Consequently, reductions in cross-entropy translate directly into shorter achievable code lengths and tighter empirical upper bounds on the source entropy rate [10].

### C. From Known Sources to Universal Compression

Thus far, we have emphasized entropy rate as an intrinsic measure of structure and as a benchmark for what lossless compression can achieve. The remaining question is constructive: *how* do practical schemes exploit structure to approach this limit? A useful way to organize classical methods is by the amount of prior knowledge they assume about the source. At one end of the spectrum are settings in which the source statistics are known, so that compression reduces to a direct translation of probability into description length. At the other end are universal schemes that operate without any prior model and must learn structure on the fly from the data stream. Between these extremes lie context-based methods that explicitly model memory through conditional probabilities, and adaptive methods that update those probabilities as new symbols arrive. Taken together, these families illustrate a recurring theme: exploiting richer structure can improve compression, but often at the cost of increased complexity, memory, and delay.

1) *Compression with a Known Source*: The simplest setting in lossless compression arises when the source statistics are fully known and symbols are generated in an iid manner. In this case, compression becomes a direct translation of probability into description length. Huffman coding provides a canonical example: given a PMF over symbols, it constructs a *prefix-free* code in which more likely symbols are assigned shorter codewords and less likely symbols receive longer ones [11], as illustrated in Fig. 1. Viewed through an information-theoretic lens, Huffman codeword lengths track the self-information  $-\log_2 p_S(s)$ , up to the integer constraints imposed by prefix-free representations.

The optimality of Huffman coding follows from this alignment with the source statistics [6]. Among all single-letter prefix-free codes, it minimizes expected description length and achieves performance within a small additive constant of the entropy. In fact, Huffman coding produces an average description length that lies within one bit of the entropy, and it matches the entropy exactly when symbol probabilities are powers of two. Although often presented as an algorithmic construction, Huffman coding can be interpreted as a predictive scheme: it anticipates which symbols are likely and allocates shorter descriptions accordingly. Its practical appeal lies in immediacy; encoding and decoding proceed sequentially with minimal overhead, making Huffman coding well suited for settings where the source statistics are stable and the latency budget is tight.

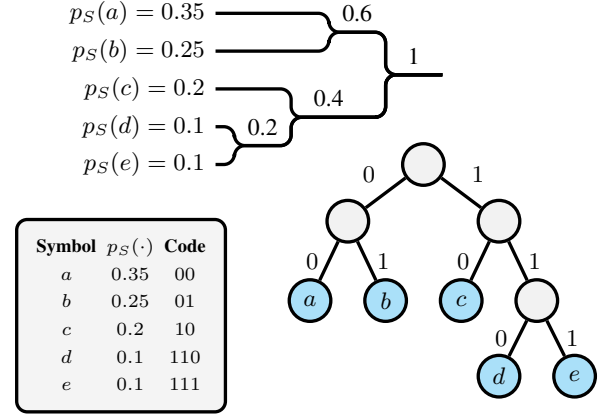


Fig. 1. Huffman coding as explicit probability-to-length assignment. Given known source statistics, a Huffman tree assigns shorter codewords to more likely symbols and longer codewords to rarer ones, producing a prefix-free bitstream that can be decoded sequentially. The figure highlights how prediction is implicit, knowing which symbols are likely determines which descriptions are shortest.

2) *Sequential Sources and Context*: Many real-world sources, including text, audio, and sensor data, exhibit strong temporal dependencies. In such sources, predictability emerges not from marginal frequencies, but from *conditional* distributions that reflect memory in the source. A classical approach is to assume a Markov structure, in which the next symbol depends on a finite number of past symbols. Compression for such sources proceeds by predicting each symbol given its context and then encoding it according to the resulting conditional probabilities.

This context-based viewpoint immediately reveals a tension. As the memory length grows, the number of conditional parameters required to describe the source increases rapidly, and reliable estimation becomes difficult for large alphabets or long contexts. To mitigate this explosion in model complexity, a rich line of work has explored variable-order models that adapt the effective memory length to the data. Context-tree methods, for example, represent conditional dependencies using trees whose depth varies across contexts, allowing short memories where appropriate and longer memories where needed [12]. These methods can significantly improve compression performance over fixed-order Markov models by balancing expressiveness and parsimony.

Despite their conceptual elegance, explicit context models still face practical tradeoffs. As dependencies extend over long horizons, the cost of model selection, estimation, and storage can become prohibitive. Implementations must balance compression efficiency against complexity, memory usage, and delay. This tension is one reason universal schemes, described below, remain so influential; they sidestep explicit probabilistic modeling while still learning structure from the data stream.

3) *Universal Compression*: The universal compression schemes abandon the assumption that source statistics are known in advance. Instead, they learn structure directly from the observed sequence, adapting as new symbols are observed [13]. This perspective is closely aligned with early work on universal coding and minimum description length (MDL),

which emphasized deep connections among compression, prediction, and statistical estimation [14].

The Lempel–Ziv family provides canonical examples of universality in practice [15], [16]. The central idea is to exploit repeated patterns without committing to an explicit probabilistic model. In the LZ77 formulation [15], repeated substrings are encoded by backward references of the form  $(d, \ell, c)$ , where  $d$  denotes the distance to a previous occurrence,  $\ell$  the match length, and  $c$  the next unmatched symbol. This approach relies on a sliding window over recent data, and treats previously observed text as an implicit dictionary, as illustrated in Fig. 2. An alternative formulation, LZ78 [16], builds an explicit dictionary of phrases as the stream progresses, encoding each new phrase as a pointer to a past phrase plus one new symbol.

Despite their structural differences, LZ77 and LZ78 embody the same principle: compression is achieved by learning and reusing structure that appears in the data. Related adaptive approaches, such as prediction by partial matching (PPM), pursue similar goals by building variable-length context models to improve predictive accuracy [2]. Remarkably, universal schemes can asymptotically achieve the entropy rate of a wide class of sources without prior knowledge of their statistical properties. Let  $L_{LZ}(\mathbf{S}_{1:n})$  denote the Lempel–Ziv description length of the sequence  $\mathbf{S}_{1:n}$ . Then

$$\lim_{n \rightarrow \infty} \frac{1}{n} L_{LZ}(\mathbf{S}_{1:n}) = H(\mathbf{S}). \quad (8)$$

The optimality of universal schemes emerges from the gradual accumulation of information about the source. In this sense, universal compression replaces parameter estimation with adaptive learning.

Yet universality comes at a cost. Unlike Huffman coding, which can be applied immediately given a known source distribution, universal schemes must *observe* structure before they can exploit it. Compression performance therefore improves with sequence length, but at the expense of increased latency and memory. These tradeoffs have been analyzed formally, with results connecting parsing, dictionary growth, and pointer representations to average decoding delay [17]. This tension between adaptability and delay is central to understanding how learning-based predictors can be integrated into compression systems, and how their performance must be weighed against practical constraints such as timeliness and synchronization.

In practice, widely deployed systems often adopt hybrid designs to balance these competing considerations. Algorithms such as DEFLATE combine Lempel–Ziv style parsing with bounded sliding windows and entropy coding, limiting memory growth and enabling more immediate decoding while retaining much of the adaptivity of universal schemes. Such designs reflect a pragmatic response to the central tension in compression: learning more structure generally improves rate, but it can also increase complexity and delay.

#### D. Predict–Then–Code as a Unifying Paradigm

The compression schemes discussed so far may appear diverse in their construction, ranging from fixed prefix codes to adaptive dictionary-based methods. At a deeper level, however,

many of these approaches can be organized around a single *predict–then–code* paradigm. In this architecture, a predictive model assigns probabilities to the next symbol (or token) given the observed history, and a coding mechanism converts those probability assignments into a compact binary representation. This modular view separates *modeling* from *coding*: the predictor supplies the *knowledge* of the source, while the coder provides an efficient way to turn that knowledge into bits. As a result, predictive accuracy becomes the primary driver of compression performance.

This perspective has deep roots in the information-theoretic tradition. Shannon’s work on the entropy of English relied on human predictors to estimate conditional probabilities, implicitly linking prediction quality to compressibility [1]. Related ideas also appeared in early engineering work on predictive coding for correlated sources, notably in studies of redundancy reduction through prediction by Elias, foreshadowing later information-theoretic treatments. Subsequent results formalized this intuition, showing that sequential probability assignments can be converted into asymptotically optimal codes whose performance reflects the quality of the underlying predictor [18]. From this standpoint, compression can be recast as a problem of learning to predict a source as accurately as possible.

Seen through this lens, even classical schemes fit naturally into the *predict–then–code* framework. Huffman coding relies on fixed probability assignments and translates them into variable-length codewords. Markov and context-based schemes refine these predictions by conditioning on past symbols. Universal methods such as Lempel–Ziv replace explicit probability estimation with adaptive mechanisms that implicitly learn which continuations are likely. In all cases, better predictions lead directly to shorter descriptions.

1) *Arithmetic Coding, the Bridge from Prediction to Compression*: At the heart of the predict–then–code paradigm lies *arithmetic coding*, a landmark development that turns explicit probabilistic prediction into near-optimal lossless compression [19], [20]. Arithmetic coding provides the missing link between assigning probabilities and realizing code lengths that closely track those probabilities. Unlike Huffman coding, which assigns integer-length codewords to individual symbols, arithmetic coding operates at the level of an entire sequence. As illustrated in Fig. 3, it represents the message by a sub-interval of  $[0, 1)$  whose width is proportional to the probability assigned to the observed prefix, and refines this interval sequentially as new symbols arrive.

This representation removes a fundamental limitation of prefix codes. Because codeword lengths must be integers, prefix codes cannot always match the ideal self-information  $-\log_2 p_S(s)$  symbol-by-symbol. Arithmetic coding sidesteps this discretization constraint. As the predictor produces conditional distributions, the coding interval is refined accordingly, and the resulting description length tracks cumulative self-information with high fidelity, even when probabilities vary over time or depend on rich contextual information. More concretely, if the predictor produces conditional probabilities  $Q(s_t | \mathbf{s}_{<t})$  along a sequence  $\mathbf{s}_{1:n}$ , then ideal arithmetic coding

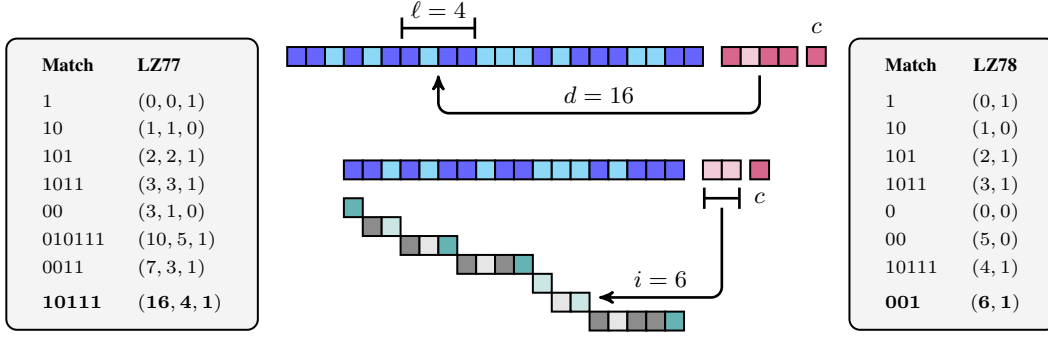


Fig. 2. Lempel-Ziv compression as adaptive pattern learning. The figure illustrates the LZ77 and LZ78 algorithms [15], [16]. In each scheme, the previously observed symbols act as an implicit dictionary. In LZ77, the encoder identifies the longest previously seen match and replaces it with a backward reference  $(d, \ell)$  together with the next unmatched symbol  $c$ . In LZ78, the encoder dynamically builds an explicit dictionary, encodes the longest match as an index into that dictionary, and appends the next symbol to form a new entry. By learning and reusing repeated patterns directly from the data, Lempel-Ziv schemes achieve universal compression without prior knowledge of the source statistics.

yields a description length that is essentially

$$L(\mathbf{s}_{1:n}) \approx - \sum_{t=1}^n \log_2 Q(s_t | \mathbf{s}_{<t}), \quad (9)$$

up to lower-order effects due to finite precision and termination conventions.

Averaged under the true source distribution  $P$ , the normalized code length converges to the cross-entropy rate between  $P$  and  $Q$ , and differs from the entropy rate of the source by a KL-divergence penalty. In this sense, arithmetic coding makes the connection between machine learning and information theory operationally transparent: reducing log-loss reduces bits. The KL divergence  $D(P||Q)$  is no longer an abstract discrepancy measure, but the expected *redundancy* paid for model mismatch.

This flexibility is what elevates arithmetic coding from an efficient coding technique to a unifying abstraction. Any model capable of producing valid conditional probability distributions (whether fixed, adaptive, or learned) can be coupled seamlessly with arithmetic coding. As a result, arithmetic coding naturally supports context-dependent models and learned predictors, and enables compression rates that approach the entropy rate of the distribution induced by the predictor. In this sense, arithmetic coding allows compression to inherit the expressive power of the underlying predictive model.

2) *Shared Models and Reliable Decoding*: An important operational consequence of predict-then-code architectures is that the encoder and decoder must share the same predictive model. Given identical models and the same past context, both sides generate the same sequence of conditional probability assignments. Decoding is then deterministic: the compressed bitstream selects the unique source sequence whose successive probability intervals contain the encoded value. This requirement is implicit in classical schemes, but it becomes explicit and central once the *model* is a large learned predictor.

As we transition to high-dimensional predictors such as LLMs, the main systems challenge shifts from designing the code to maintaining synchronization. Arithmetic coding largely resolves the coding side of the problem; lossless recovery hinges instead on keeping the encoder and decoder

in lockstep on *state*. In practice, this means agreeing not only on the model weights and tokenizer, but also on details that are easy to overlook: context initialization, prompts or side information, numerical precision, normalization conventions, and termination rules. Even small discrepancies can compound over time, causing the decoder’s reconstructed context to drift and eventually diverge from the encoder’s state. In short, learned compression replaces *designing clever codes* with the engineering discipline of ensuring exact reproducibility.

Viewed through this lens, arithmetic coding is not merely an implementation detail, but a conceptual bridge between classical information theory and modern learning-based compression. Once compression is understood as prediction followed by arithmetic coding, improvements in compression rate admit a clear interpretation: they correspond directly to improvements in predictive accuracy. This perspective allows compression performance to serve as a rigorous, quantitative proxy for how well a model captures the structure of a source.

### III. THE EXPERIMENT: LETTING LLMs COMPRESS TEXT

With the predict-then-code paradigm in place, the experiment is conceptually simple: treat a modern LLM as a next-token probability engine, convert those probabilities into a bitstream, and measure the resulting compression rate. What is new is not the *principle*, but the *regime*. Earlier text compressors relied on hand-crafted statistical structures (fixed or variable-order context models), while universal methods learned patterns implicitly from the data stream. Neural compressors then made the predictive step explicit by training sequence models to assign conditional probabilities and coupling them with near-optimal coders [3].

LLMs push this program much further. They provide markedly sharper conditional distributions over substantially longer contexts, making it possible to revisit a classic question with modern tools: when prediction improves dramatically, how far can lossless text compression be pushed, and what does that imply for empirical bounds on the entropy rate of English [21]? In this section, we walk through the experiment from the top down: what changed with LLMs, how probabilities become bits, what must be shared to decode reliably, and what the resulting compression rates tell us.



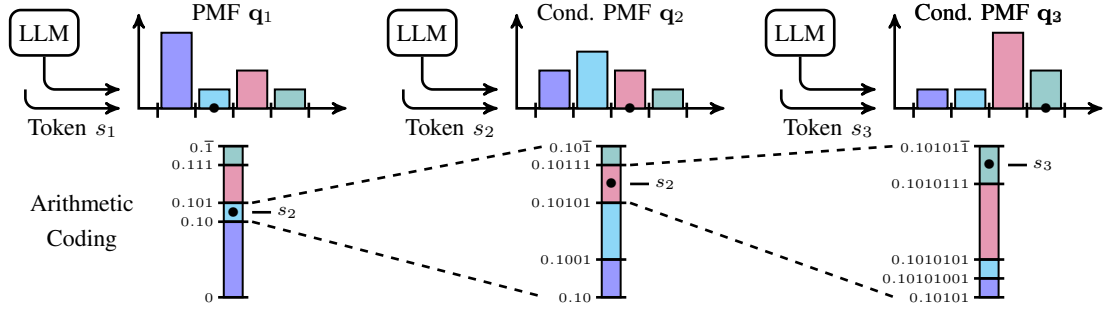


Fig. 3. Arithmetic coding: turning probabilities into bits. As symbols arrive, their conditional probabilities successively refine an interval on  $[0, 1]$  whose width equals the probability of the observed prefix. As the interval shrinks, the leading bits in its binary representation become fixed (shared by every point in the current interval) and can be emitted immediately, enabling streaming transmission. The resulting bitstream length tracks cumulative log-loss.

#### A. From Neural Predictors to LLMs: What Changed?

A central lesson of *neural compression* is that lossless compression performance is governed primarily by predictive accuracy, because arithmetic-style coders can convert sequential probability assignments into near-optimal bitstreams. DeepZip provided an early concrete illustration of this idea for text: an RNN-based predictor assigned next-symbol probabilities, and those probabilities were then coded to yield a learned compressor [3]. Subsequent work refined this picture through broader empirical evaluations and modern neural architectures, with performance commonly reported in bits per character (bpc) as a direct proxy for predictive log-loss [22].

Transformers and modern LLMs, depicted in Fig. 4, push the practical frontier in three intertwined ways that matter directly for compression. First, scaling and training data reduce cross-entropy, meaning that the model assigns higher probability to the observed continuation on real text. Second, attention enables conditioning on long contexts, allowing the predictor to exploit dependencies that extend well beyond fixed-order Markov assumptions. Third, tokenization provides a practical interface between raw characters and the model: compression is performed over tokens that represent variable-length strings, while final rates can still be reported per character through normalization. Taken together, these changes make LLMZip, the combination of an LLM predictor with arithmetic coding, a natural continuation of the prediction-plus-coding program, but in a regime where the predictor is substantially stronger and the compression performance is correspondingly superior [21].

#### B. From Conditional Probabilities to Bits

The LLMZip setup makes the predict-then-code pipeline fully explicit. A tokenizer maps a character string  $s$  into a token sequence  $\mathbf{x} = (x_1, \dots, x_{N_T})$  over a vocabulary  $\mathcal{X}$ , where  $N_T$  is the number of tokens. Given a context window of length  $M$ , a language model assigns at token position  $i$  a conditional distribution

$$q_i(x) = \Pr(X_i = x | X_{i-1} = x_{i-1}, \dots, X_{i-M} = x_{i-M}) \quad (10)$$

where  $x \in \mathcal{X}$ . The encoding mechanism then turns these probabilities into bits.

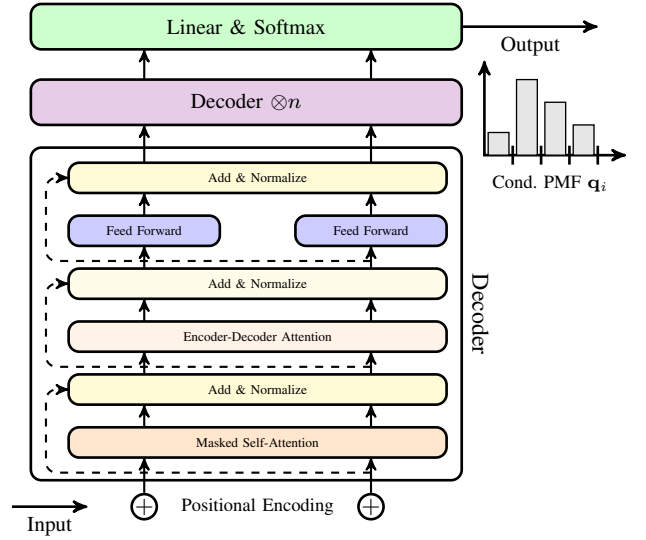


Fig. 4. Next-token prediction with a transformer. Attention allows the model to condition on a long context and assign probabilities to possible continuations. In compression settings, these probabilities become a direct resource: more accurate predictions translate into fewer bits after compression.

There are two closely related ways to leverage the model output. In an *ideal* predict-then-code implementation, one codes the tokens directly using an entropy coder driven by  $q_i(\cdot)$ , yielding a description length that tracks the cumulative log-loss,

$$L(\mathbf{x}) \approx - \sum_{i=1}^{N_T} \log_2 q_i(x_i), \quad (11)$$

up to standard lower-order effects due to finite precision and termination conventions. Alternatively, the LLMZip construction maps each realized token  $x_i$  to its *rank* under the model's sorted distribution, producing a rank sequence  $r_1, \dots, r_{N_T}$  that becomes highly compressible when the predictor is strong, and then compresses that rank sequence using a general-purpose compressor (e.g., `zlib`) [21]. Both viewpoints emphasize the same operational message: when the predictor assigns high probability to the observed continuation, fewer bits are needed to describe it.

A useful feature of this pipeline is that it connects directly to entropy-rate estimation. Under stationarity and ergodicity as-

sumptions, the Shannon–McMillan–Breiman theorem provides an almost-sure characterization of entropy rate, and results such as [18] justify using predictive log-loss as an asymptotic upper bound. To compare rates across representations, LLMZip makes the tokenization step explicit by relating the token entropy rate  $H(\mathbf{X})$  to the character entropy rate  $H(\mathbf{S})$  through the average number of characters per token,  $\mathbb{E}[B]$ :

$$H(\mathbf{S}) = \frac{H(\mathbf{X})}{\mathbb{E}[B]}. \quad (12)$$

This yields an empirical upper bound of the form

$$H(\mathbf{S}) \leq \frac{\lim_{N_T \rightarrow \infty} -\frac{1}{N_T} \sum_{i=1}^{N_T} \log_2 q_i(X_i)}{\mathbb{E}[B]}, \quad (13)$$

which is reported in bpc [18], [21].

Recent results help justify this setup from a classical source-modeling viewpoint. For sources that are well approximated by a finite-memory process, the key object for coding is the conditional law  $\Pr(X_i | X_{i-1}, \dots, X_{i-M})$ , and compression depends on learning it accurately. Recent work shows that transformer predictors can learn broad classes of Markov sources and approximate their finite-order conditional structure [23], [24], [25], [26]. While natural language is not Markov of fixed order, these findings support the empirical observation that larger context windows sharpen  $q_i(\cdot)$  and improve predict-then-code rates.

### C. Shared Models and Synchronization

As mentioned in Section II-D2, the predict-then-code construction is lossless only if the decoder can reproduce *exactly* the same probability assignments used by the encoder. In practice, this means that both sides must share the same tokenizer, the same model weights, and compatible numerical conventions for computing and normalizing probabilities. They must also agree on the context initialization (and any prompts or side information), since the evolving context determines the conditional distributions  $q_i(\cdot)$  at every token position.

This requirement is conceptually straightforward but operationally subtle. Even small discrepancies such as different tokenizer versions, differences in floating-point behavior, mismatched probability normalization, or even the number of layers offloaded to the graphical processing unit can cause decoding to diverge, after which the reconstructed context drifts and the errors compound. This clarifies what is easy in principle (prediction-driven coding) and what becomes delicate in practice (exact reproducibility, synchronization). It also exposes an emerging theme for modern learned compressors: as predictive models become larger and more expressive, the main systems challenge shifts from designing codes to ensuring that both ends of the link remain perfectly aligned on the predictor and its evolving state.

A related practical constraint is that lossless implementation typically requires access to the model’s *full* conditional distribution over the vocabulary at each step. Some large-model deployments and commercial application programming interfaces (APIs) return probabilities only for the most likely tokens (or omit calibrated probabilities entirely), which is sufficient

for text generation but not for general-purpose entropy coding. Arithmetic coding and closely related schemes rely on having a complete, normalized distribution (or an equivalent mechanism that assigns a probability to *every* possible next token), since omitted tail mass cannot be ignored without changing the implied code lengths and breaking decodability. From a systems perspective, this means that the feasibility of LLM-based compression depends not only on model quality, but also on whether the inference interface exposes distributions with the fidelity required for coding.

### D. Empirical Performance and Entropy-Rate Estimation

The empirical question is whether the improved predictive accuracy of LLMs translates into improved lossless compression and tighter entropy estimates when paired with a suitable coding mechanism. Using LLaMA-7B as the predictor, LLMZip reports substantially lower bpc values than earlier results, both as an entropy upper bound (via average log-loss) and as an achieved compression rate under concrete coding constructions [21]. For example, on a 1 MB slice of the text8 dataset, the reported asymptotic upper-bound estimate reaches approximately 0.709 bpc (with a 511-token context), while a token-by-token coding construction achieves about 0.845 bpc under the stated implementation choices [21]. Applying the same methodology to a 100 KB Project Gutenberg text (after text8-style preprocessing) yields higher bpc values, consistent with distribution shift, while still improving over previously reported neural baselines [21], [22].

Table I summarizes representative results, contrasting LLMZip (which couples an LLM predictor with an entropy coder) against widely used classical compressors and selected reference bounds reported in the literature. These numbers convey two complementary messages. First, stronger prediction lowers cross-entropy, which tightens entropy upper bounds and reduces achievable compression rates. Second, the gap between an entropy upper bound (log-loss) and an implemented compression rate reflects practical overheads: finite-precision arithmetic, termination conventions, and the particulars of the coding construction (e.g., rank+zlib versus token-by-token arithmetic-style coding).

Another practical observation, implicit in Table I and visible empirically, is that context length matters. Increasing the context window  $M$  improves bpc by exposing more structure to the predictor, but it also increases inference cost; in the reported experiments, inference time scaled roughly linearly with  $M$  [21]. Taken together, these results make the role of the LLM concrete: it is the component that moves the assigned conditional probabilities closer to the true source conditionals, thereby reducing the KL mismatch penalty and translating directly into fewer bits.

## IV. EXTENSIONS AND DESIGN TRADEOFFS: DELAY, DISTORTION, AND ADAPTATION

The preceding sections cast compression as a predict-then-code pipeline and used LLMZip as a concrete instantiation. In many networked settings, however, performance is not captured by compression rate alone. Latency constraints make



TABLE I  
COMPARATIVE COMPRESSION PERFORMANCE OF LLMZIP BASED ON LLAMA2 AND ARITHMETIC CODING, TO PRIOR ART AND EXISTING BOUNDS.  
RESULTS ARE REPORTED IN BITS PER CHARACTER (BPC)

| Dataset   | LLMZip       | ZPAQ | paq8 | zlib | Upper bound [1] | Lower bound [1] | Upper bound [18] | Upper bound [22] |
|-----------|--------------|------|------|------|-----------------|-----------------|------------------|------------------|
| text8     | <b>0.693</b> | 1.52 | 1.4  | —    | 1.4             | 0.6             | 1.2              | 1.05             |
| Gutenberg | <b>0.774</b> | —    | —    | 2.55 | 1.4             | 0.6             | 1.2              | 1.05             |

the temporal profile of code lengths as important as their mean, while perceptual or task-driven applications motivate lossy representations that trade rate for controlled distortion. This section highlights two complementary extensions. We first revisit lossless coding through a queueing lens, emphasizing how load and burstiness affect delay in streaming links. We then turn to a lossy formulation, where a shared predictor can be coupled with a variable-slope sequential rule that adaptively enforces a distortion target while maintaining low delay. Ultimately, these considerations are intertwined, since allowing controlled distortion can reduce the rate and variability of the encoded stream and thereby improve timely delivery.

#### A. Timely Communication: When Latency Matters

Compression is often treated as a rate-minimization problem: the objective is to drive the average bits per symbol toward the entropy rate. In real-time communication settings, however, *timeliness* is often as critical as efficiency. When a compressor emits variable-length descriptions into a constant-rate channel of  $R$  bits/second, encoded symbol  $i$  induces a service time  $S_i = \ell_i/R$ . From a queueing-theoretic perspective, end-to-end delay depends not only on the mean length  $\mathbb{E}[\ell_i]$  (the system *load*) but also on the variability of  $\ell_i$  (the *burstiness*). This is the basic message of classical queueing formulas such as Pollaczek–Khinchine: as utilization approaches unity, mean waiting time grows rapidly and is amplified by the second moment of the service-time distribution.

This lens highlights a familiar tension in the design of text compressors. Huffman coding prioritizes immediacy, offering a simple, symbol-by-symbol mapping with negligible buffering and low algorithmic overhead. However, integer-length constraints and limited context can leave significant rate overhead on sources with memory, increasing the average load presented to the queue. At the other extreme, universal schemes such as LZ77/LZ78 approach the entropy rate for broad classes of stationary ergodic sources without an explicit probabilistic model, but they do so by parsing the stream into repeated and novel phrases. This introduces two operational penalties that matter in streaming. First, there is a *cold-start* effect: useful matches require accumulated history, so compression remains inefficient until a sufficiently large window has been filled; and any block reset discards that context, re-triggering the penalty. Second, match lengths fluctuate, producing highly variable output (pointer–literal bursts) that translate into high service-time variance and increased jitter.

Predict–then–code architectures based on LLMs offer a compelling middle ground when endpoint compute is available. Coupling a strong predictor with arithmetic coding is

naturally sequential: tokens can be encoded and transmitted as they arrive, without waiting for long phrase discovery or large block formation. At the same time, arithmetic coding converts sharp conditional distributions into code lengths that closely track  $-\log_2 q(x_i|x_{<i})$ , so the achieved rate approaches the predictor’s cross-entropy. Relative to dictionary-based parsing, this mechanism can reduce load while avoiding some of the most extreme burstiness associated with long-match outputs. The price is shifted to inference: the system spends computation and energy at the endpoints to save bits on the link and to stabilize the service process.

In short, Huffman enables immediacy, Lempel–Ziv emphasizes universality and asymptotic optimality, and LLM-based predict–then–code provides a favorable rate–latency tradeoff when near real-time communication is the objective and endpoint compute is an acceptable resource to trade for fewer bits and smoother delivery.

#### B. Extension to Lossy Compression

The universal compression framework reviewed in Section II-C3 admits a well-established extension to the lossy case, in which the aim is to reconstruct each source sequence up to a given distortion constraint. Given a source sequence  $\mathbf{s}_{1:n}$  and alternative  $\hat{\mathbf{s}}_{1:n}$ , we let  $d(\mathbf{s}_{1:n}, \hat{\mathbf{s}}_{1:n})$  denote the distortion incurred by approximating  $\mathbf{s}_{1:n}$  with  $\hat{\mathbf{s}}_{1:n}$ . In *fixed-slope* universal lossy data compression, one selects  $\hat{\mathbf{s}}_{1:n}$  as the solution of the minimization problem [27]

$$\hat{\mathbf{s}}_{1:n} = \arg \min_{\tilde{\mathbf{s}}_{1:n}} \{L_{\text{LZ}}(\tilde{\mathbf{s}}_{1:n}) + \lambda d(\mathbf{s}_{1:n}, \tilde{\mathbf{s}}_{1:n})\} \quad (14)$$

where  $\lambda$  is the slope parameter. For stationary and ergodic sources satisfying (8), one can show that, as  $n \rightarrow \infty$ , the rate necessary to represent  $\hat{\mathbf{s}}_{1:n}$  and the incurred per-symbol distortion  $d(\mathbf{s}_{1:n}, \hat{\mathbf{s}}_{1:n})/n$  converge to the point  $(R_\lambda, D_\lambda)$  on the rate–distortion curve at which the slope of the rate–distortion function equals  $-\lambda$ . Unfortunately, the minimization of (14) is in general challenging, since the objective is not additive. Replacing the Lempel–Ziv encoder with an LLM, as in Section III, does not resolve this difficulty, even in settings where the distortion metric is additive.

A first attempt to side-step this issue was recently proposed in [28], where the authors introduce a sequential (or zero-delay) approximation to (14) that operates as follows. At each time  $i$ , the symbol  $\hat{s}_i$  used to approximate  $s_i$  is selected as the solution of (see Fig. 5)

$$\hat{s}_i = \arg \min_{\tilde{s}_i} \{-\log_2 q_i(\tilde{s}_i) + \lambda_i d(s_i, \tilde{s}_i)\}. \quad (15)$$

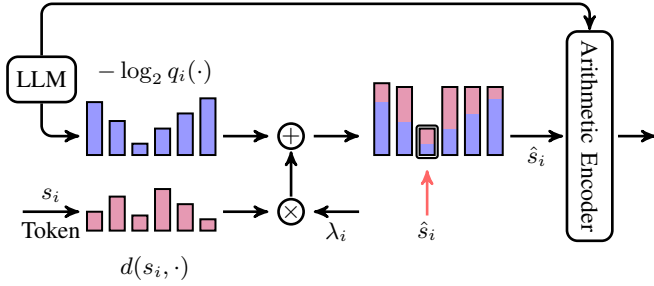


Fig. 5. This diagram illustrates LLM-aided sequential lossy compression operating according to the variable-slope rule (15). Token substitution is admissible and guided through the local distortion metric  $d(\cdot, \cdot)$ .

In analogy with (10),  $q_i(\tilde{s}_i)$  denotes the conditional distribution produced by the LLM, given the current reconstructed context window (a substring of  $\hat{s}_{<i}$ ). Importantly, the context is a substring of  $\hat{s}_{<i}$  and not of  $s_{<i}$ , since the latter is available only at the encoder. We also allow the slope to depend on the time index  $i$ . Specifically, for a target per-symbol distortion  $D$ , the slope parameter  $\lambda_i$  is adapted according to the following online conformal prediction rule [29]:

$$\lambda_{i+1} = \lambda_i + \eta (d(s_i, \hat{s}_i) - D) \quad (16)$$

where  $\eta > 0$  is a step size and  $\lambda_0 > 0$ . For this setting, the authors provide universal (i.e., sequence- and predictor-independent) finite-sample bounds on the distortion. In particular, they show that the distortion is upper-bounded by  $D$  for sufficiently large sequence length  $n$ . Approaches in this spirit are well suited to scenarios in which timeliness is critical, including systems operating under strict delay constraints or streaming settings that trade rate for controlled distortion.

## V. WHEN DOES AN LLM COMPRESSOR MAKE SENSE?

A candid caveat is that LLM-based compression may be deemed *unfair* if evaluated as a standalone file utility. Modern language models are memory-intensive and computationally hungry, often requiring gigabytes of memory and substantial inference compute. From the viewpoint of classical source coding, this shifts cost from the transmitted bitstream to the shared predictor itself. In this sense, LLM-based compression is closer in spirit to *amortized* coding than to the traditional setting where the compressor is a lightweight algorithm with a negligible footprint.

This perspective is also closer to a two-part description-length view, where one must account for the cost of describing both the predictive *model* and the *data conditioned on that model*. From this angle, downloading a multi-billion-parameter model solely to compress a single short text file would be preposterous: the *model overhead* would dwarf any savings in bits by orders of magnitude. A relevant regime for LLM-based compression is one in which the model is *already deployed* as shared infrastructure. As LLMs become embedded in operating systems, browsers, and mobile hardware, it becomes plausible that compatible models are readily available at both the source and the destination. If the predictor is already present on-device for generative or analytical tasks, then the

marginal cost of using it for compression is dominated by inference latency and energy rather than model dissemination. In this regime, the pertinent comparison is not “LLMZip versus gzip” on a one-off basis, but rather: *how many bits can we save on an ongoing stream when the predictor is already there?*

Two concrete scenarios make the amortization explicit. First, in streaming communication, one may accept additional compute at the endpoints to reduce the sustained communication rate, especially when bandwidth is scarce or expensive. Here, the LLM plays the role of a rich shared source model, and the savings accrue continuously over time. Second, in large-scale archiving, the one-time cost of model installation can be amortized over a large corpus. If an organization compresses and stores terabytes of text (or repeatedly transmits similar data), even modest reductions in bits per character can compound into meaningful savings in storage and bandwidth. This framing also points toward high-dimensional modalities such as images and video. If shared learned predictors can capture the typicality of natural scenes, human motion, or multimodal streams more effectively than hand-crafted priors, then the model becomes a reusable asset: a shared prior that can lower communication and storage costs at scale.

In short, LLM-based compression is most compelling when the model cost is either sunk (because the model is already deployed) or amortized (because the data volume is large), and when communication or storage costs are sufficiently important that spending compute to save bits is a rational systems tradeoff.

## VI. INTERPRETATION AND IMPLICATIONS

The empirical performance of LLMZip is not merely a statement about improved engineering. It is also a measurement of how predictable natural language is when viewed through the lens of next-token conditional distributions. In particular, lossless compression with an LLM probes the local continuation structure of text rather than any form of *mind-reading*. The model is rewarded for assigning high probability to the next token given the preceding context, and much of this signal reflects shared regularities: grammar, idioms, discourse markers, collocations, and genre conventions. However, modern LLMs also exploit structure at the level of *content* and *intent*. When context makes the topic and communicative goal clear, the model can leverage broad world knowledge, common scripts (e.g., explanations, narratives, arguments), and familiar document templates to concentrate probability on a narrower set of continuations. In this sense, strong compression indicates not only that language has redundant surface form, but also that human-generated text often follows predictable higher-level plans once a discourse trajectory is established. Equivalently, once topic and intent are inferred, the conditional entropy of future tokens given the past can drop sharply, because many continuations become implausible.

This clarification helps reconcile compression with the widely observed *analysis and synthesis* behavior of LLMs. Such capabilities can be interpreted as inferring implicitly a latent plan or discourse state (the outline of an argument,

the steps of a procedure, the likely resolution of a query) and then generating text consistent with that structure. From the standpoint of predict-then-code, better latent-structure inference translates into sharper conditional distributions over multi-token spans, which can reduce description length even when the relevant regularities are semantic or pragmatic rather than purely syntactic.

A striking aspect is that a model of moderate scale can act as a broadly effective predictor across writers and domains. This suggests a universality of the linguistic *codebook* rather than a universality of the person. Much of the predictive burden is shared across humans because language is a coordination protocol: we share conventions, rhetorical templates, and communal priors about what comes next in common situations. Individuality appears as smaller deviations (style, preferred constructions, favored metaphors, topic priors). This highlights a subtle point in interpreting learning. Humans are not necessarily predictable in their thoughts, but words are not independent. Language is redundant, and encoding rich internal states into shared linguistic form imposes structure that can be exploited for prediction and compression.

This picture admits a clean Bayesian reading. The LLM encodes a broad prior over language, shaped by training, and each observed token acts as evidence that concentrates the posterior over likely continuations. When topic and intent become clear from context, the posterior sharpens and conditional entropy drops, exactly the mechanism behind the rate improvements reported in Section III-D. In this view, the compression rate achieved by predict-then-code measures how well the model’s approximate posterior tracks the true conditional distribution: the gap is the KL mismatch penalty, and tighter posteriors translate directly into fewer bits. This also reinforces the two-part description-length perspective of Section V: the model parameters encode the prior, and the arithmetic-coded bitstream encodes the data given that prior. Improvements in compression thus have a dual interpretation, as evidence that the learned prior captures genuine statistical structure in the source.

These observations connect naturally to the notion of a *language fingerprint*. Stylometry is real: authors differ in function-word usage, punctuation patterns, sentence length distributions, syntactic preferences, hedging, register, idioms, and topic mixtures. At the same time, such signals are not reliable biometrics. Style shifts with audience, stakes, medium, fatigue, and topic, and small samples can be misleading. What a context window often reveals is not identity but *likely next moves*: how a writer tends to qualify claims, what kinds of examples they reach for, what they emphasize, and which rhetorical transitions they employ. In compression terms, such attributes act as side information: once they are implicit in context, they can reduce uncertainty about what is said next.

#### A. Co-Evolution of Language and AI

If learned predictors become embedded in everyday interfaces, then language itself may adapt to the presence of these predictors. This evolution can pull in opposing directions. On one hand, widespread exposure to model-suggested phrasing

may increase homogenization of style and register, reinforcing shared conventions and potentially increasing compressibility. On the other hand, users and communities may adopt *anti-predictable* signatures such as idiosyncratic turns of phrase, deliberate novelty, or jargon to preserve voice, emphasize authenticity, or simply resist template-like prose. Either way, the predictor is no longer a passive observer of a fixed source. It becomes part of the communication ecology, and compression performance becomes a coupled phenomenon: it reflects the model’s predictive power and the way people adapt their writing when they expect a model in the loop.

#### B. Meaning-Preserving Rewriting as Preprocessing

A separate, practical direction is to improve compressibility not by changing the coder, but by changing the *surface form* of the message while preserving meaning. Natural language admits many paraphrases with the same semantics but different predictability under a given model. A meaning-preserving rewrite can increase the probability of the realized continuation under the shared predictor, thereby reducing description length under predict-then-code. At a high level, this suggests a two-stage pipeline: (i) transform the message into a semantically equivalent canonical form that aligns with common constructions, and (ii) compress the canonical form using a shared predictive model and arithmetic coding.

This idea is best viewed as a controlled, opt-in preprocessing step rather than a universal replacement for ordinary writing. It raises three technical questions. First, what constraints define *meaning preserving* in a way that is operational and verifiable? Second, how should one trade stylistic fidelity against compressibility, especially when voice and tone carry information beyond propositional content? Third, how should the rewrite be communicated or agreed upon so that semantic fidelity is maintained, and individual stylistic preference reintroduced at the decoder? These questions connect compression to classical themes in semantics, paraphrase, and canonicalization, and they point toward hybrid systems in which learned predictors are used not only to code text, but also to normalize it in ways that reduce redundancy under shared conventions.

#### C. Beyond Token-by-Token Coding: Batch Reveal

Standard LLM-based compression is naturally sequential: the encoder reveals one token at a time and the model updates its conditional distribution. This matches the operational structure of arithmetic coding and yields clean connections to cross-entropy. However, sequential reveal is not the only possible interaction pattern between encoder and decoder. One direction is *batch reveal*, in which the encoder transmits a compact side information that disambiguates a *set* of future tokens at once. Conceptually, this trades a fine-grained stream of small updates for fewer, larger updates that resolve uncertainty over longer horizons. Such schemes could reduce overheads associated with frequent coder renormalization, termination conventions, and finite-precision effects, and they may offer better rate-latency tradeoffs in packetized systems.

More broadly, the presence of a shared predictor enables a spectrum of interactive protocols. The encoder can choose

what to reveal and when, conditioned on the model’s uncertainty. For instance, when the model assigns high probability mass to a small set of continuations, the encoder can transmit only a small index identifying the correct branch. When uncertainty is diffuse, the encoder may fall back to a more explicit representation. This viewpoint suggests that “LLM compression” is not a single algorithm but a design space of protocols that exploit a shared conditional distribution, with sequential arithmetic coding as one particularly clean operating point.

#### D. Sequential Text Versus Partially Ordered Inputs in Vision

Text presents a canonical sequential source: the input is totally ordered, and next-token prediction aligns directly with a causal factorization of the data distribution. Vision and multimodal data do not share this property. Images are not naturally ordered in a unique way, and many vision-language architectures operate on sets or grids of features where locality and permutation structure matter. This difference has two implications for compression. First, the notion of *next* element is a modeling choice: one must decide an ordering (raster scan, patches, multiscale trees) or adopt coders that operate over sets with adaptive context. Second, long-range dependencies in images are often spatial rather than temporal, so the most useful *context* may be multi-resolution and non-causal.

In this setting, the analogue of predict-then-code is to design a factorization that exposes conditional structure efficiently: choose an ordering (or a hierarchy) that makes the conditional distributions sharp, then apply arithmetic-style coding to those conditionals. The success of LLMZip for text suggests that the limiting factor is not the coder but the quality of the conditional model. For high-dimensional modalities, the challenge shifts to defining a representation and factorization under which learned predictors can provide calibrated probabilities with manageable computation and reliable synchronization. Viewed this way, LLMZip is less a new coder than a measurement instrument: it quantifies how much structure (syntactic, semantic, and pragmatic) a shared predictor can extract and reuse.

#### E. Outlook

The convergence of large-scale learned predictors with classical coding theory opens a rich design space in which model quality, computational cost, and communication latency can be traded off in principled ways. As language models grow more capable and efficient, and as analogous predictors mature for vision, speech, and multimodal data, the predict-then-code paradigm is poised to become a practical and unifying framework for next-generation compression and communication.

#### REFERENCES

- [1] C. E. Shannon, “Prediction and entropy of printed English,” *Bell Syst. Tech. J.*, vol. 30, no. 1, pp. 50–64, Jan. 1951.
- [2] J. G. Cleary and I. H. Witten, “Data compression using adaptive coding and partial string matching,” *IEEE Trans. Commun.*, vol. 32, no. 4, pp. 396–402, Apr. 1984.
- [3] M. Goyal, K. Tatwawadi, S. Chandak, and I. Ochoa, “DeepZip: Lossless data compression using recurrent neural networks,” in *Data Compress. Conf. (DCC)*, Snowbird, UT, USA, 2019, pp. 575–575.
- [4] C. E. Shannon, “A mathematical theory of communication,” *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, Jul. 1948.
- [5] F. Jelinek, *Markov Source Modeling of Text Generation*. Springer, 1985, pp. 569–591.
- [6] T. M. Cover, *Elements of information theory*. John Wiley & Sons, 1999.
- [7] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, “Distributed representations of words and phrases and their compositionality,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 26, Stateline, Nevada, USA, 2013.
- [8] F. Jelinek, R. L. Mercer, L. R. Bahl, and J. K. Baker, “Perplexity—a measure of the difficulty of speech recognition tasks,” *J. Acoust. Soc. Amer.*, vol. 62, no. S1, pp. S63–S63, 1977.
- [9] L. Bahl, J. Baker, P. Cohen, A. Cole, F. Jelinek, B. Lewis, and R. Mercer, “Automatic recognition of continuously spoken sentences from a finite state grammar,” in *IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP)*, vol. 3, Tulsa, OK, USA, 1978, pp. 418–421.
- [10] G. Delétang, A. Ruoss, P.-A. Duquenne, E. Catt, T. Genewein, C. Matern, J. Grau-Moya, L. K. Wenliang, M. Aitchison, L. Orseau, M. Hutter, and J. Veness, “Language modeling is compression,” in *Int. Conf. Learn. Represent. (ICLR)*, Vienna, Austria, 2024.
- [11] D. A. Huffman, “A method for the construction of minimum-redundancy codes,” *Proc. IRE*, vol. 40, no. 9, pp. 1098–1101, Sep. 1952.
- [12] F. M. J. Willems, Y. M. Shtarkov, and T. Tjalkens, “The context-tree weighting method: Basic properties,” *IEEE Trans. Inf. Theory*, vol. 41, no. 3, pp. 653–664, May 1995.
- [13] N. Merhav and M. Feder, “Universal prediction,” *IEEE Trans. Inf. Theory*, vol. 44, no. 6, pp. 2124–2147, 1998.
- [14] J. Rissanen, “Universal coding, information, prediction, and estimation,” *IEEE Trans. Inf. Theory*, vol. 30, no. 4, pp. 629–636, 1984.
- [15] J. Ziv and A. Lempel, “A universal algorithm for sequential data compression,” *IEEE Trans. Inf. Theory*, vol. 23, no. 3, pp. 337–343, Jul. 1977.
- [16] —, “Compression of individual sequences via variable-rate coding,” *IEEE Trans. Inf. Theory*, vol. 24, no. 5, pp. 530–536, Sep. 1978.
- [17] P. Jacquet and W. Szpankowski, “Asymptotic behavior of the Lempel-Ziv parsing scheme and digital search trees,” *Theor. Comput. Sci.*, vol. 144, no. 1–2, pp. 161–197, Jun. 1995.
- [18] T. M. Cover and R. C. King, “A convergent gambling estimate of the entropy of English,” *IEEE Trans. Inf. Theory*, vol. 24, no. 4, pp. 413–421, Jul. 1978.
- [19] J. Rissanen and G. G. Langdon, “Arithmetic coding,” *IBM J. Res. Dev.*, vol. 23, no. 2, pp. 149–162, 1979.
- [20] I. H. Witten, R. M. Neal, and J. G. Cleary, “Arithmetic coding for data compression,” *Commun. ACM*, vol. 30, no. 6, pp. 520–540, Jun. 1987.
- [21] C. S. K. Valmeekam, K. Narayanan, D. Kalathil, J.-F. Chamberland, and S. Shakkottai, “LLMZip: Lossless text compression using large language models,” *arXiv preprint arXiv:2306.04050*, 2023.
- [22] S. Lutati, I. Zimerman, and L. Wolf, “Focus your attention (with adaptive IIR filters),” in *Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Singapore, 2023.
- [23] R. Zhou, C. Tian, and S. Diggavi, “Transformers learn variable-order markov chains in-context,” in *Adv. Neural Inf. Process. Syst. Workshop*, Sydney, Australia, 2025.
- [24] A. V. Makkuvu, M. Bondaschi, A. Girish, A. Nagle, M. Jaggi, H. Kim, and M. Gastpar, “Attention with Markov: A framework for principled analysis of transformers via markov chains,” *arXiv preprint arXiv:2402.04161*, 2024.
- [25] C. Ekbote, M. Bondaschi, N. Rajaraman, J. D. Lee, M. Gastpar, A. V. Makkuvu, and P. P. Liang, “What one cannot, two can: Two-layer transformers provably represent induction heads on any-order markov chains,” *arXiv preprint arXiv:2508.07208*, 2025.
- [26] E. Edelman, N. Tsilivis, B. L. Edelman, E. Malach, and S. Goel, “The evolution of statistical induction heads: In-context learning markov chains,” *Advances in neural information processing systems*, vol. 37, pp. 64 273–64 311, 2024.
- [27] E.-H. Yang, Z. Zhang, and T. Berger, “Fixed-slope universal lossy data compression,” *IEEE Trans. Inf. Theory*, vol. 43, no. 5, pp. 1465–1476, May 1997.
- [28] M. Zecchin, U. K. Ganesan, G. Durisi, P. Popovski, and O. Simeone, “Prediction-powered communication with distortion guarantees,” *arXiv preprint arXiv:2509.24373*, Sep. 2025.
- [29] I. Gibbs and E. J. Candès, “Conformal Inference for Online Prediction with Arbitrary Distribution Shifts,” *J. Mach. Learn. Res.*, vol. 25, no. 162, pp. 1–36, May 2024.